Тема 14: Обектно-ориентирано програмиране. Подтипов и параметричен полиморфизъм. Множествено наследяване

Анотация:

- Виртуални функции и подтипов полиморфизъм.
    - Динамично свързване.
    - Абстрактни методи и класове.
    - Параметричен полиморфизъм.
    - Множествено наследяване.
- Шаблони на функция и на клас
- Масиви от обекти и от указатели към обекти.

Полиморфизъм

Полиморфизмът представлява един от фундаменталните принципи в обектно-ориентираното програмиране (ООП), който позволява дефиниране на унифициран интерфейс за различни типове обекти, осигурявайки гъвкавост и разширяемост на кода.

Видове полиморфизъм

Полиморфизмът бива два основни вида: статичен (compile-time) и динамичен (run-time). Разликата между тях се състои в момента на избор на конкретната функция, която ще бъде извикана - по време на компилация или по време на изпълнение.

- Статичен (compile-time) полиморфизъм
    - Известен още като overloading
        - Function overloading
            - Функции със същото име, но с различен брой или тип параметри.
        - Operator overloading
            - Позволява предефиниране на стандартни оператори, като `+`, `==`, `[]` и други, за да се адаптират към потребителски типове данни.
        - Параметричен полиморфизъм (templates)
            - Осъществява се чрез шаблони (templates) на функции и класове.
            - Позволява създаването на функции или класове, които работят с различни типове без повторение на кода.
            - Използва се за реализиране на обобщени алгоритми и структури от данни.

```
template<typename T>
T max(T a, T b) {
    return (a > b) ? a : b;
}
```

- Изборът на подходяща функция се извършва по време на компилация.

```
#include <iostream>

class Base {
public:
    void f() const  {
            std::cout << "A::f()\n";
    }
};

class Derived : public Base {
public:
    void f() const {
            std::cout << "B::f()\n";
    }
};

void func(const Base& obj) {
    obj.f();
}

int main() {
    Derived* bPtr = new Derived();
    Base* aPtr = bPtr;

    // Early binding - Compile time
    func(*aPtr); // Base::f()
    func(*bPtr); // Base::f()
    // Въпреки че aPtr сочи към обект от тип Derived, методът f() не е
виртуален,
    // затова се извиква версията от класа Base (ранно свързване).
}
```

Това се случва, защото функцията func приема параметър от тип const Base&. По време на компилация компилаторът знае само статичния тип на обекта (Base) и тъй като методът f() не е виртуален, свързването е статично и се избира Base::f().

- Динамичен (run-time) полиморфизъм
    - Реализира се чрез наследяване и виртуални функции
    - Когато извикваме функция чрез указател или препратка към базов клас, се изпълнява функцията на обекта, към който сочи, дори ако той е от производен клас.
    - Изборът на функция се извършва по време на изпълнение (run-time).

```
#include <iostream>

class Base {
public:
virtual void f() const {
        std::cout << "A::f()\n";
}
};

class Derived : public Base {
public:
void f() const override {
    std::cout << "B::f()\n";
}
```

```
};

int main() {
    Base* ptrs[2];
    ptrs[0] = new Base(); //Base pointer to Base class object
    ptrs[1] = new Derived(); //Base pointer to Derived class object

    //Dynamic/Late Binding - Runtime
    ptrs[0]->f(); // Извиква Base::f(), защото обектът е от тип Base
    ptrs[1]->f(); // Извиква Derived::f(), защото обектът е от тип Derived

    // Освобождаване на паметта
    delete ptrs[0];
    delete ptrs[1];
}
```

Виртуални функции

Виртуалните функции осигуряват механизма за реализиране на динамичен (run-time) полиморфизъм. Те се декларират в базовия клас с ключовата дума virtual и позволяват поведение, което зависи от действителния тип на обекта, а не от типа на указателя или референцията

- Динамичното свързване се осъществява чрез виртуална таблица (vTable) и указател към нея (vPtr), съхраняван във всеки обект с виртуални функции.
- Виртуалните функции не могат да бъдат static, тъй като изискват достъп до конкретен обект.
- За да се осигури правилно унищожаване на обекти чрез указатели към базов клас, деструкторът трябва да бъде виртуален.

Спецификатори:

- override - изрично указва, че функцията предефинира виртуална такава от базовия клас; помага за избягване на грешки.
- final - забранява по-нататъшно предефиниране на дадена виртуална функция или наследяване от даден клас.

Нека да разгледаме следната йерархия:

```
#include <iostream>

class Base {
public:
  virtual void f() const {
    std::cout << "Base::f()\n";
  }

  virtual void g() const {
    std::cout << "Base::g()\n";
  }

  void nonVirtual() const {
    std::cout << "Base::nonVirtual()\n";
  }
};
```

```
class FirstDerived : public Base {
public:
  void f() const override {
    std::cout << "FirstDerived::f()\n";
  }

  void g() const override {
    std::cout << "FirstDerived::g()\n";
  }

  virtual void h() const {
     std::cout << "FirstDerived::h()\n";
  }
};

class SecondDerived : public FirstDerived {
public:
  void f()const override {
    std::cout << "SecondDerived::f()\n";
  }
};

class ThirdDerived : public SecondDerived {
  public:
    void h() const override {
      std::cout << "ThirdDerived::h()\n";
    }
};
```

и да създадем обекти от всеки един от тях:

```
int main() {
   Base baseObject;
   FirstDerived firstDerivedOject;
   SecondDerivedsecondDerivedObject;
   ThirdDerived thirdDerivedObject;

   Base* p = nullptr;

   p = &baseObject;
   p->f(); //static call to Base::f()
   p->g(); //static call to Base::gl()
   p->nonVirtual();  //static call to Base::nonVirtual()

   std::cout << "----------" << std::endl;

   p = &firstDerivedOject;
   p->f(); // Dynamic binding - FirstDerived::f()
   p->g(); // Dynamic binding - FirstDerived::g()

   std::cout << "----------" << std::endl;

   p = &secondDerivedObject;
   p->f(); // Dynamic binding - SecondDerived::f()
   p->g(); // Dynamic binding - FirstDerived::g()

   std::cout << "----------" << std::endl;

   p = &thirdDerivedObject;
   p->f(); // Dynamic binding - SecondDerived::f()
   p->g(); // Dynamic binding - FirstDerived::g()
   p->h(); // Dynamic binding - ThirdDerived::h()
}
```

На указател към базов клас (p) може да бъде присвоен адресът на обект от всеки клас, участващ в йерархията на наследяване — включително обекти от производни класове. Благодарение на механизма на динамичен полиморфизъм, чрез този указател могат да бъдат извиквани виртуални функции, като по време на изпълнение (run-time) се определя коя конкретна реализация на функцията трябва да бъде извикана в съответствие с действителния (динамичен) тип на обекта.

За да реализира това поведение, компилаторът генерира специални вътрешни структури за управление на виртуалните функции. По време на компилация, за всеки клас в йерархията, който съдържа поне една виртуална функция (или наследява такава), се създава т.нар. виртуална таблица (vtable). Това е масив от указатели към съответните виртуални функции.

Всяка инстанция на клас, който има виртуални функции, съдържа скрит указател (vptr), който сочи към виртуалната таблица на съответния клас. При извикване на виртуална функция чрез указател или препратка към базов клас, този указател (vptr) се използва за намиране на „правилната“ функция, т.е. тази, която съответства на реалния тип на обекта, а не на типа на указателя.

Така се осигурява коректно поведение на виртуалните функции и се постига динамично свързване (late binding), което е в основата на полиморфизма в обектно-ориентираното програмиране.

Абстрактни класове и чисто виртуални функции

Чисто виртуалната функция е `виртуална функция`, чиято декларация в базовия клас завършва на `= 0`. Тази специална нотация има двойна роля:

- Освобождава базовия клас от задължението да предоставя незабавна реализация и по този начин го превръща в абстрактен клас.
- Задължава всеки конкретен клас наследник да предостави своя собствена реализация (override) на тази функци

```
class Shape {
public:
    virtual void draw() const = 0; // чисто виртуална функция
    virtual ~Shape() = default;
};
```

- Клас, съдържащ поне една чисто виртуална функция, се нарича абстрактен.
- Забранява създаването на инстанции. С други думи, от абстрактни класове не могат да се създават обекти директно.
- Те служат като интерфейс за производните класове, които са задължени да реализират всички чисто виртуални функции, за да станат инстанциируеми.

```
class Animal {
public:
    virtual void speak() const = 0; // абстрактна функция
    virtual ~Animal() = default;    // виртуален деструктор
};

class Dog : public Animal {
```

```
public:
    void speak() const override {
        std::cout << "Woof!\n";
    }
};
```

Този пример демонстрира как абстрактният базов клас Animal дефинира поведение (speak()), което се реализира конкретно в производния клас Dog.

Уточнение: Чисто виртуална функция може да има своя реализация/тяло, обаче:

- Тялото на функцията трябва да бъде дефинирано извън декларацията на класа

```
class Shape {
public:
    virtual void draw() const = 0; // чисто виртуална функция
    virtual ~Shape() = default;
};

void Shape::draw() {
    // Да, чисто виртуална функция с реализация!
    std::cout << "Това е обща логика за рисуване от базовия клас Shape." <<
std::endl;
}
```

- Основната причина е да се предостави обща, споделена функционалност, от която класовете наследници могат да се възползват, ако желаят.
    - Класът Shape си остава абстрактен. Маркировката = 0 все още е в сила и все още не можете да създадете обект от тип Shape.
    - Наследниците все още са длъжни да я предефинират. За да стане един наследник конкретен клас, той трябва да има свое override на draw().
    - Наследникът може да извика базовата реализация: Вътре в своята предефинирана функция наследникът може изрично да извика реализацията от базовия клас.

```
class Circle : public Shape {
public:
    void draw() override {
        // 1. Извикваме общата функционалност от базовия клас
        Shape::draw();

        // 2. Добавяме специфичната за кръга логика
        std::cout << "По-конкретно, рисувам кръг." << std::endl;
    }
};
```

Множествено наследяване

В CPP се позволява множествено наследяване – т.е. производен клас може да има повече от един директен базов клас:

```
class A { ... };
class B { ... };
class C : public A, public B { ... }; // множествено наследяване
```

Предимства:

- Позволява комбиниране на функционалности от различни базови класове;
- Улеснява повторната употреба на код от независими източници.

Диамантен проблем (Diamond Problem)

Множественото наследяване може да доведе до двусмислици в случаите, когато един и същ базов клас се среща повече от веднъж в наследствената йерархия.

```
//Indicate problem - Пример без виртуално наследяване

#include <iostream>

class SuperClass {
public:
  SuperClass() {
    std::cout << "SuperClass default constructor is called\n";
  }
};

class A : public SuperClass {
public:
  A() {
    std::cout << "A default constructor is called\n";
  }
};

class B : public SuperClass {
public:
  B(){
      std::cout << "B default constructor is called\n";
  }
};

class C : public A, public B {
public:
  C() {
    std::cout << "C default constructor is called\n";
  }
};

int main() {
    C obj;
}
```

Output:

```
SuperClass default constructor is called
A default constructor is called
SuperClass default constructor is called //<------------
B default constructor is called
C default constructor is calle
```

Конструкторът на SuperClass се извиква два пъти – веднъж чрез A и веднъж чрез B. Това означава, че обектът obj има две отделни копия на членовете на SuperClass. Ако достъпим например SuperClass::x, компилаторът няма да знае кое копие имаме предвид – възниква ambiguous behavior.

За да избегнем този проблем, използваме виртуално наследяване, което гарантира, че в

наследника ще съществува само едно споделено копие на общия базов клас.

```
#include <iostream>

class SuperClass {
public:
  SuperClass() {
    std::cout << "SuperClass default constructor is called\n";
  }
};

class A : virtual public SuperClass {
public:
  A() {
    std::cout << "A default constructor is called\n";
  }
};

class B : virtual public SuperClass {
public:
  B(){
    std::cout << "B default constructor is called\n";
  }
};

class C : public A, public B {
public:
    C() {
        std::cout << "C default constructor is called\n";
    }
};

int main() {
    C obj;
}
```

Output:

```
SuperClass default constructor is called
A default constructor is called
B default constructor is called
C default constructor is called
```

Вече SuperClass се създава само веднъж, което елиминира двусмислието и спестява памет.

Пояснение: Когато използваме виртуално наследяване, съществуват следните особености:

- Въпреки че базовите класове (напр. A, B) могат да извикват конструктори с параметри, само най-долният производен клас (в случая C) трябва да избере конструктора, с който да инициализира виртуалния базов клас;
- Ако SuperClass няма конструктор по подразбиране, трябва изрично да го извикаме в списъка за инициализация на C.

```
class SuperClass {
public:
    SuperClass(int x) { std::cout << "SuperClass(" << x << ")\n"; }
};

class A : virtual public SuperClass {
public:
    A() : SuperClass(0) { std::cout << "A()\n"; } // няма ефект!
```

```
};

class B : virtual public SuperClass {
public:
    B() : SuperClass(1) { std::cout << "B()\n"; } // няма ефект!
};

class C : public A, public B {
public:
    C() : SuperClass(42), A(), B() {
        std::cout << "C()\n";
    }
};
```

Ще бъде извикан само SuperClass(42) – инициализацията чрез A и B ще бъде игнорирана!

При виртуално наследяване редът на извикване на конструкторите е:

- Конструкторите на виртуалните базови класове (извикани от най-производния клас).
- Конструкторите на невиртуалните базови класове (в реда на тяхното деклариране).
- Конструкторите на член-данните (в реда на тяхното деклариране).
- Тялото на конструктора на самия клас.

Шаблони на функция и на клас

Шаблоните (templates) са механизъм в езика C++, който позволява създаване на обобщен код. Те осигуряват параметричен полиморфизъм, позволявайки на функции и класове да работят с различни типове данни, без да се налага дублиране на кода. С други думи, шаблонът е една дефиниция, която компилаторът използва, за да генерира конкретни реализации въз основа на зададените типове по време на компилация.

Шаблони на функции

Шаблон на функция представлява обобщена дефиниция на функция, при която типът на параметрите не е фиксиран, а се дефинира чрез параметри на шаблона.

```
template <typename T>
T функция_име(T параметър1, T параметър2);
```

- template - ключова дума, указваща, че следва шаблон.
- typename T - определя параметър на шаблона, който ще бъде заместен с конкретен тип при извикване на функцията.

Може да се използва и ключовата дума class вместо typename - двете са взаимозаменяеми в този контекст.

```
template <typename T>
T sum(const T& a, const T& b) {
    return a + b;
}
```

Тази функция работи с всеки тип, който поддържа операцията + — например int, double, std::string и др.

```
int main() {
    std::cout << sum<int>(2, 3) << std::endl;          // Явно указване на типа
    std::cout << sum(2.5, 3.1) << std::endl;           // Неявно определяне на
типа (type deduction)
}
```

Компилаторът генерира конкретна реализация на функцията чрез подмяна на T с подадения тип. Този процес се нарича инстанциране на шаблон.

Шаблони на класове

Шаблон на клас позволява дефиниране на клас, който може да оперира с произволен тип данни. Подобно на функциите, вместо да създаваме отделни класове за различни типове (IntStack, DoubleStack и т.н.), можем да използваме един шаблонен клас Stack.

```
template <typename T>
class ИмеНаКлас {
    T данни;
public:
    void set(const T& value);
    T get() const;
};

// Пример:
template <typename T>
class Box {
private:
    T value;
public:
    void set(const T& val) { value = val; }
    T get() const { return value; }
};

// Използване:
int main() {
    Box<int> intBox;
    intBox.set(42);
    std::cout << intBox.get() << std::endl;

    Box<std::string> strBox;
    strBox.set("Hello");
    std::cout << strBox.get() << std::endl;
}

// При компилация се създават конкретни реализации на класа за типа int,
std::string и т.н.
```

Параметри на шаблона

Шаблоните могат да имат:

- Типови параметри (typename T)
- Нестойностни параметри (например цяло число):

```
template <typename T, int size>
class Array {
    T data[size];
};
```

Предимства на шаблоните

- Позволяват писане на обобщен код, който може да работи с различни типове.
- Подобряват реюзабилността и гъвкавостта на кода.
- Осигуряват type safety, тъй като типовете се проверяват по време на компилация.
- Позволяват създаване на контейнери (като std::vector, std::map и др.) в стандартната библиотека на C++ (STL), които са базирани на шаблони.

Ограничения и особености

- Ако даден тип не поддържа операция, използвана в шаблона (например +, ==), ще възникне грешка при компилация.
- Може да се създават специализации на шаблон — т.е. специално поведение за конкретен тип.

Дефинициите на шаблони (templates) не могат да бъдат разделени от декларациите им в отделни .h и .cpp файлове, следвайки традиционния модел на компилация. Това ограничение произтича директно от фундаменталната природа на шаблоните и начина, по който компилаторът работи с тях чрез процес, наречен инстанциране

Компилаторът трябва да генерира конкретна версия на кода (инстанциране) за всеки отделен тип, с който се използва шаблонът. За да извърши това инстанциране, компилаторът се нуждае от достъп до цялата дефиниция (тялото) на шаблона в момента на извикването му. Ако дефиницията е скрита в отделен .cpp файл, компилаторът не може да генерира нужния код, което по-късно води до грешка за "липсващ символ" на етапа на свързване (linking). Поради тази причина стандартната практика е цялата имплементация на шаблона да се поставя в хедър файла (.h), за да е видима за всички компилационни единици, които я използват.

Масиви от обекти и от указатели към обекти.

Можем да създаваме масиви не само от базови типове (като int, char и др.), но и от потребителски дефинирани типове (обекти от класове). Съществуват различни варианти за създаване на такива масиви:

- Статични масиви от обекти
- Динамични масиви от обекти
- Масиви от указатели към обекти (както статични, така и динамични)

Всеки от тези варианти има свои специфики по отношение на създаване, използване и управление на паметта.

Стеков масив от обекти (Плътно разположение в паметта)

```
class A {
public:
    A() { std::cout << "Constructor\n"; }
    ~A() { std::cout << "Destructor\n"; }
};
```

```
int main() {
    A arr[5];
}
```

Обяснение:

- Създаваме масив с фиксирана дължина 5 от обекти от тип A.
- Компилаторът извиква конструктора по подразбиране 5 пъти – по веднъж за всеки елемент.
- Когато програмата напусне обхвата на функцията main, за всеки от елементите автоматично се извиква деструктор.
- Паметта се управлява автоматично (на стека).

Характеристики:

- Обектите са плътно разположени в паметта.
- Не може да се използва различен конструктор за отделни елементи.
- Размерът е фиксиран по време на компилация.

Динамичен масив от обекти

```
class A {
public:
    A() { std::cout << "Constructor\n"; }
    ~A() { std::cout << "Destructor\n"; }
};

int main() {
    A* arr = new A[5];   // динамичен масив от 5 обекта
    delete[] arr;        // освобождаване на паметта
}
```

Обяснение:

- Създаваме масив от обекти в динамичната памет чрез new[].
- Компилаторът автоматично извиква конструктора по подразбиране за всеки елемент.
- При извикване на delete[]:
    - Извиква се деструктор за всеки елемент.
    - Освобождава се цялата заделена памет.

Характеристики:

- Размерът на масива може да бъде променлив (определя се в runtime).
- Всички обекти използват един и същи конструктор.
- Изисква delete[] (а не просто delete), за да се извикат коректно всички деструктори.

Масиви от указатели

Подходът за съхранение на масив от указатели е фундаментално различен от съхранението

на масив от обекти. Вместо да разполага обектите в непрекъснат блок памет, масивът съдържа единствено адреси (указатели). Самите обекти могат да се намират на произволни места в динамичната памет (heap) или дори на стека. Това разделяне на колекцията от нейните елементи се нарича индиректност (indirection) и е източникът на огромна гъвкавост и мощ, но и на по-голяма отговорност за програмиста.

Фундаментална гъвкавост

Масивът от указатели позволява няколко нива на гъвкавост, невъзможни при масивите от обекти:

- Разнородно съхранение
    - Указателите могат да сочат към обекти, създадени по различно време и на различно място – някои в динамичната памет (new), други на стека.
- Различни конструктори
    - Всеки елемент може да бъде създаден с различен, параметризиран конструктор, позволявайки индивидуална инициализация на всеки обект.
- Празна позиция
    - Един елемент на масива може да бъде nullptr, което ясно индикира "празен слот" в колекцията.
    - При масив от обекти, всеки елемент е напълно конструиран обект.

Проблемът "Нарязване на обекти" (Object Slicing)

Масив от указатели към базов клас (`Base* arr[N]`) решава този проблем. Един указател от тип `Base*` може легитимно да сочи към адрес на обект от тип `Derived` без никаква загуба на информация. Това позволява на една колекция да управлява хетерогенен (разнороден) набор от обекти, стига всички те да споделят общ базов клас

Полиморфен контейнер е всяка структура от данни (като масив, `std::vector` или списък), която е проектирана да съхранява хетерогенна колекция от обекти с различен тип, които обаче споделят общ базов клас. Тъй като обектите не могат да се съхраняват директно в контейнера поради проблема с "нарязването", тази функционалност се постига чрез съхранение на указатели или референции към базовия клас.

Забележка: Както споменахме по-горе - За да работи полиморфизмът, е необходимо да се спазват две ключови правила.

- Полиморфното поведение се активира чрез ключовата дума virtual пред методите в базовия клас, което позволява на извиквания като arr[i]->draw() да изпълнят правилната версия на функцията по време на изпълнение.
- Деструкторът на базовия клас също да бъде virtual, за да се гарантира коректното извикване на деструкторите на наследниците при изтриване с delete arr[i] и да се предотвратят течове на памет.

Предимства при управление на ресурси

Тъй като масивът съдържа само малки по размер указатели, а не самите "тежки" обекти, това

води до значителни оптимизации:

- Ефективно преоразмеряване (Efficient Resize)
    - Ако трябва да увеличите капацитета на колекцията, е необходимо да заделите нов, по-голям масив от указатели, да копирате само указателите (бърза, "плитка" операция) и да изтриете стария масив от указатели.
    - Самите обекти не се местят, копират или пресъздават.
- Ефективна размяна (Efficient Swap)
    - Размяната на съдържанието на две огромни колекции, управлявани по този начин, се свежда до размяната на два указателя (A**).
    - Това е моментална операция с константна сложност O(1).